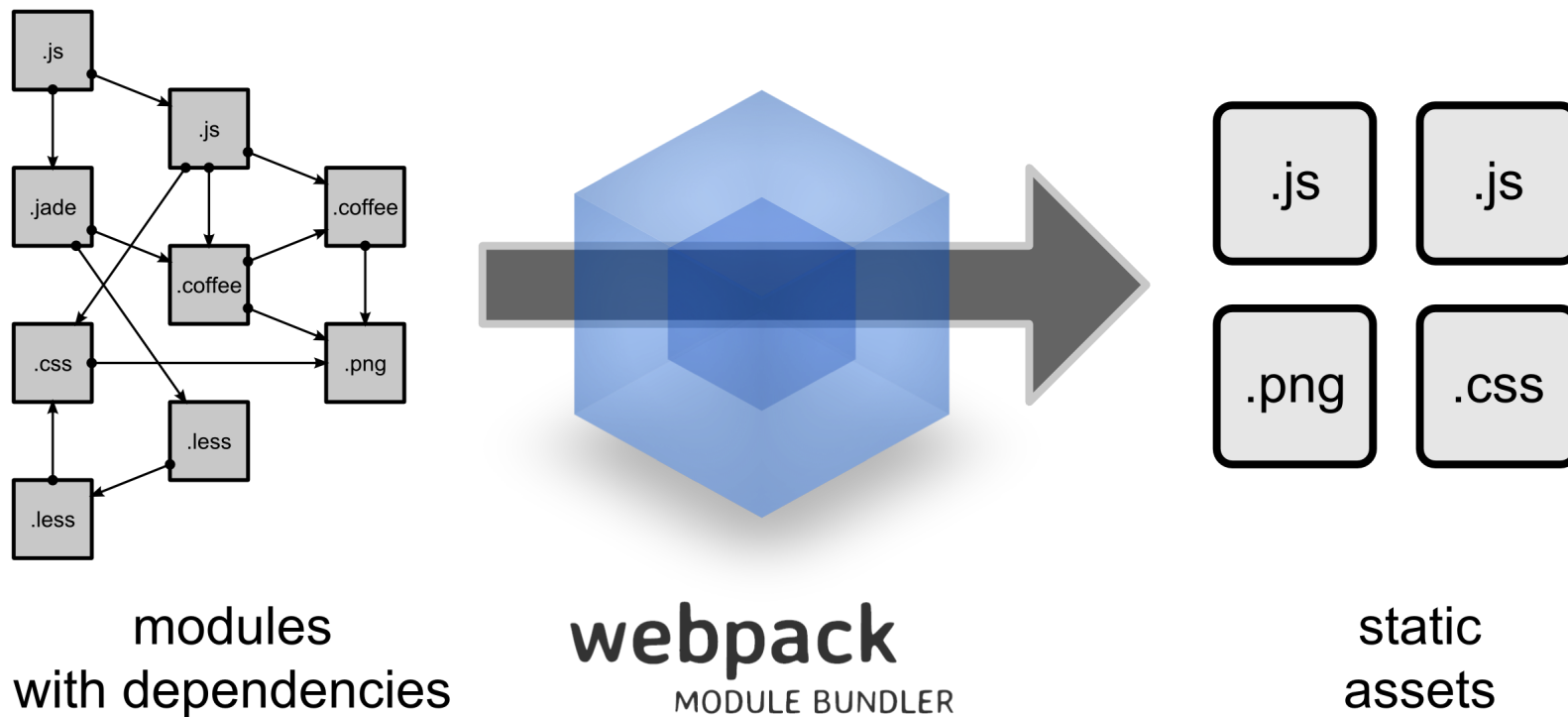


webpack 入门

什么是 webpack?



一个针对现代 JavaScript 程序的模块打包器
(A module bundler for modern JavaScript applications)

```
//index.html
<!DOCTYPE html>
<html lang='en'>
<head>
  <meta charset='UTF-8' />
  <title>Document</title>
  <link rel='stylesheet' href='./index.css' />
</head>
<body>
</body>
<script type="text/javascript" src="./index.js"></script>
<script type="text/javascript" src="./other.js"></script>
</html>
```

```
//index.js
function hello() { var hello = 'hello'; }
//other.js
function hello() { var world = 'world'; }
```

```
//index.css
.hello {
  width: 100px;
}
.world {
```

JavaScript 的模块化方案

1. 命名空间模式
2. 依赖注入
3. amd
4. umd
5. CommonJS
6. ESModule

延伸阅读：[一文回顾 JavaScript 模块演化简史](#)

CommonJS

```
const hello = require('./hello')  
const say = hello + 'world'  
module.exports = say
```

非 node.js 环境下

```
/* 1 */
/*!*****!\
!*** ./hello.js ***!
\*****/
/***/ function(module, exports, __webpack_require__) {
    const hello = require('./hello')
    const say = hello + 'world'
    module.exports = say/***/ },
```

ESModule

```
import hello from ('./hello')  
const say = hello + 'world'  
export default say
```

区别

1. CommonJS 能够动态加载，而 ESMModule 则为静态加载
2. import 的模块名只能是字符串常量，导入的值也是不可变对象
3. import 具有提升效果

webpack 支持哪些模块机制?

js

1. ESMODULE(import & export)
2. CommonJS(require & module.exports)
3. AMD(define & require)

non-js

1. css/sass/less(@import)
2. image(url(...)) & `<img src=...>`

```
const happyThreadPool = HappyPack.ThreadPool({ size: os.c
```

```
module.exports = {  
  entry: './src/index.js', // 入口文件  
  output: {  
    filename: 'bundle/index.js' // 打包输出路径  
  },  
  resolve: {  
    modules: [ // 模块查找路径  
      'node_modules'  
    ]  
  },  
  module: {  
    rules: [ // loader 解析规则  
      {  
        test: /\.css$/,  
        loader: 'happypack/loader',  
        options: {  
          id: 'css'  
        }  
      }  
    ]  
  },  
  plugins: [ // webpack 插件  
    new HappyPack({  
      id: 'css',  
      threadPool: happyThreadPool,
```

核心概念

1. Entry
2. Output
3. Loaders
4. Plugins

Entry

```
entry: './src/index.js', // 入口文件
```

程序入口

Output

```
output: {  
  filename: 'bundle/index.js' // 打包输出路径  
},
```

Loaders

1. 一切皆模块
2. 仅理解 JavaScript
3. 将指定格式文件通过 JavaScript 处理

配置 Loaders

```
module: {  
  rules: [  
    { test: /\.css$/, use: ['style-loader', 'css-loader'] },  
    { test: /\.png$/, loader: 'url-loader' },  
  ]  
}
```

```
import './index.less'
```

1. less-loader 将 less 转成 css
2. css-loader 将 url(xxx.png) 转成 url(require(./xxx.png))
3. style-loader 将 css 内容加到 style 标签内

Note:

css-loader 将 css 中的 url(xxx.png) 这类图片提取出来，转成 url(require(./xxx.png)) 的形式，即 webpack 中一切都是模块

loader example

```
// raw-loader  
module.exports = function(content) {  
    this.cacheable && this.cacheable();  
    this.value = content;  
    return "module.exports = " + JSON.stringify(content)  
}
```


another loader example

style-loader

Plugins

配置 Plugins

```
plugins: [ // webpack 插件
  new HappyPack({
    id: 'css',
    threadPool: happyThreadPool,
    loaders: [ 'style-loader', 'raw-loader' ],
    verbose: true,
    verboseWhenProfiling: true
  }),
]
```

1. Compiler - The compiler object represents the fully configured Webpack environment.
2. Compilation - A compilation object represents a single build of versioned assets.

plugin example

```
function HelloCompilationPlugin(options) {}

HelloCompilationPlugin.prototype.apply = function(compiler) {
  compiler.plugin('done', function() {
    console.log('Hello World!');
  });
  // Setup callback for accessing a compilation:
  compiler.plugin("compilation", function(compilation) {

    // Now setup callbacks for accessing compilation steps
    compilation.plugin("optimize", function() {
      console.log("Assets are being optimized.");
    });
  });
};

module.exports = HelloCompilationPlugin;
```

```
function MyPlugin() {}
```

```
MyPlugin.prototype.apply = function(compiler) {  
  compiler.plugin('emit', function(compilation, callback)  
  
    // Explore each chunk (build output):  
    compilation.chunks.forEach(function(chunk) {  
      // Explore each module within the chunk (built input)  
      chunk.modules.forEach(function(module) {  
        // Explore each source file path that was included  
        module.fileDependencies.forEach(function(filepath)  
          // we've learned a lot about the source structure  
        });  
      });  
    });  
  
    // Explore each asset filename generated by the chunk  
    chunk.files.forEach(function(filename) {  
      // Get the asset source for each file generated by the chunk  
      var source = compilation.assets[filename].source();  
    });  
  });  
  
  callback();  
});  
};
```

从 compiler 看编译流程

1. compile(编译任务)
2. after-compile(编译完成)
3. emit(即将准备生成文件)
4. after-emit(生成文件之后)
5. 其它任务(after-plugins, after-resolvers)

compiler emit

```
compiler.plugin('emit', function(compilation, callback) {  
  // Explore each chunk (build output):  
  compilation.chunks.forEach(function(chunk) {  
    // Explore each module within the chunk (built inputs)  
    chunk.modules.forEach(function(module) {  
      // Explore each source file path that was included  
      module.fileDependencies.forEach(function(filepath) {  
        // we've learned a lot about the source structure  
      });  
    });  
  });  
  
  // Explore each asset filename generated by the chunk  
  chunk.files.forEach(function(filename) {  
    // Get the asset source for each file generated by  
    var source = compilation.assets[filename].source();  
  });  
});  
  
  callback();  
});
```

生成的打包文件

--->

webpack --profile

--->

扫我

